EVOASTRA VENTURES

Internship Project

Project Title:

SUPPLY CHAIN DEMAND PREDICTION SYSTEM

PROJECT DOCUMENTATION

1. Project Overview

The Supply Chain Demand Prediction System is an end-to-end machine learning application designed to forecast product demand based on key supply chain and operational factors. The primary objective of this project is to assist businesses in making data-driven decisions related to inventory management, logistics planning, and resource allocation.

In earlier phases, advanced machine learning models such as Random Forest, XGBoost, and Neural Networks were developed and evaluated using the DataCo supply chain dataset. In Phase 4, these trained models are integrated into a complete application with a user-friendly interface and backend support, enabling real-time predictions.

The system allows users to input important features such as order quantity, shipping delay, profit margin, and seasonal indicators to generate predicted sales values. The application also provides a dashboard to visualize prediction statistics and usage trends.

This phase focuses on transforming the machine learning pipeline into a fully functional application by integrating:

- A frontend interface (Streamlit) for user interaction
- A backend API (FastAPI) for handling requests and model inference
- Pre-trained machine learning models for prediction
- A local database to store prediction history and analytics

Overall, the project demonstrates the practical implementation of machine learning in a real-world supply chain scenario, combining predictive modeling with application development and system integration.

2. System Architecture

The system follows a modular architecture that integrates frontend, backend, machine learning models, and data storage to enable real-time demand prediction. Each component is designed to perform a specific role while interacting seamlessly with the others.

2.1 Overall Workflow

The complete flow of the system can be described as follows:

User Input (Frontend)

- Request sent to Backend API
- Data Processing & Model Selection
- Model Inference (Prediction)
- Output Displayed to User
- Prediction Stored in Database

2.2 Components of the System

1. Frontend (Streamlit):
The frontend is built using Streamlit and provides an interactive user interface. It allows users to input required features, select machine learning models, and view prediction results and analytics through a dashboard.

2. Backend API (FastAPI):
The backend is implemented using FastAPI, which handles incoming requests from the frontend. It processes user input, performs validation, and connects with the machine learning models to generate predictions.

3. Machine Learning Models:
The system uses pre-trained models including Random Forest, XGBoost, and Neural Networks. These models are stored as serialized files (.pkl and .h5) and are loaded during runtime to perform predictions.

4. Database (Local Storage):
A lightweight local database (predictions.db) is used to store user inputs and prediction results. This data is utilized to generate dashboard metrics such as total predictions, average predicted sales, and peak values.

5. Deployment Layer (Docker):
The backend service is containerized using Docker, ensuring consistent deployment and execution across different environments. This allows the system to be easily deployed and scaled.

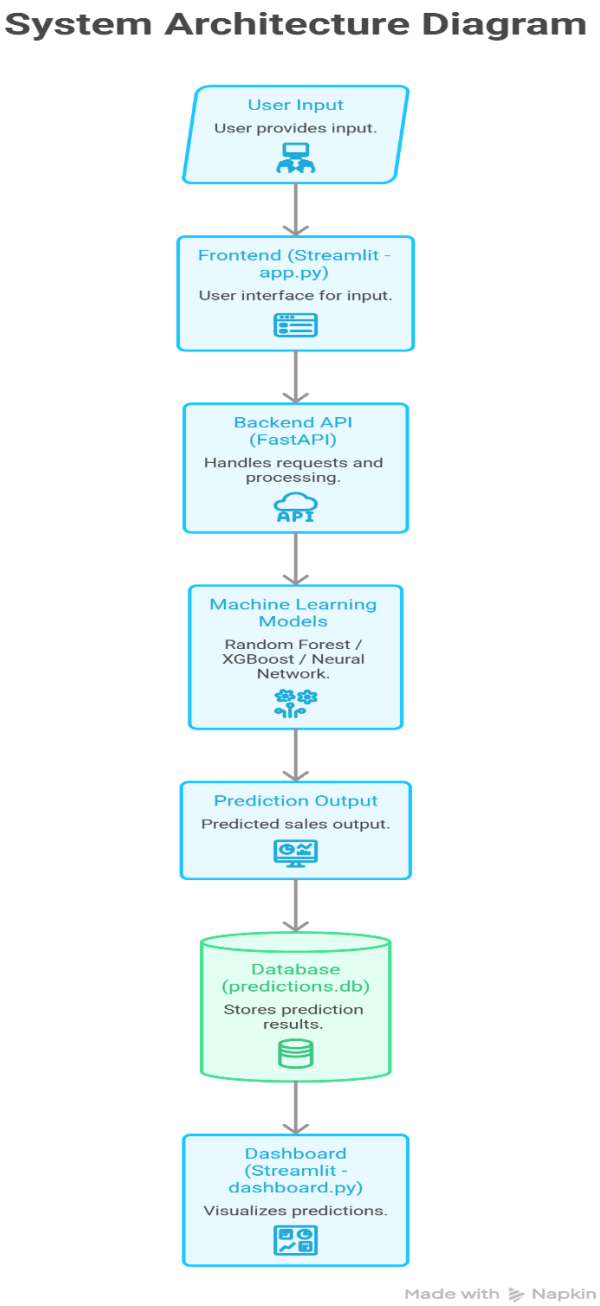
2.3 Key Characteristics of the Architecture

- Modular Design: Each component operates independently but is fully integrated
- Scalability: Backend and frontend can be scaled separately
- Real-time Processing: Immediate prediction based on user input
- Data Persistence: Predictions are stored for analytics and monitoring

3. Architecture Diagram

The following diagram illustrates the overall system architecture of the Supply Chain Demand Prediction System, showing the interaction between the frontend, backend API, machine learning models, and data storage components.

The user interacts with the system through a Streamlit-based frontend interface. The input data is sent to the FastAPI backend, where it is processed and passed to the selected machine learning model for prediction. The predicted output is then returned to the frontend and displayed to the user. Additionally, the prediction results are stored in a local database, which is used to generate dashboard analytics and insights.



4. Frontend Implementation

The frontend of the Supply Chain Demand Prediction System is developed using Streamlit, which provides a simple and interactive web-based interface for user interaction. The frontend enables users to input data, select machine learning models, and visualize prediction results along with analytical insights.

The frontend is primarily implemented using two main files: app.py and dashboard.py.

4.1 app.py

The app.py file serves as the main entry point of the application. It is responsible for handling user interaction and controlling the overall flow of the application.

Key functionalities include:

- Providing a user interface for entering input features such as order quantity, shipping delay, profit margin, and order details
- Allowing users to select different machine learning models (Random Forest, XGBoost, Neural Network)
- Processing user input and triggering prediction requests
- Displaying prediction results in real time
- Implementing basic navigation and user interface elements

4.2 dashboard.py

The dashboard.py file is responsible for displaying analytical insights and historical prediction data. It enhances the usability of the system by providing a visual representation of model performance and usage trends.

Key functionalities include:

- Displaying key performance indicators such as total predictions, average predicted sales, and peak predicted values
- Showing the most frequently used machine learning model
- Visualizing prediction trends and statistics
- Enhancing user understanding through a structured dashboard interface

4.3 Key Features of the Frontend

- Interactive and user-friendly interface
- Real-time prediction generation
- Support for multiple machine learning models
- Integration with backend API for processing
- Dashboard visualization for analytics
- Lightweight and easy to deploy

5. API Integration

The system uses a FastAPI-based backend to enable communication between the frontend interface and the machine learning models. The API acts as a bridge that processes user input, performs model inference, and returns prediction results to the frontend.

5.1 Role of FastAPI

FastAPI is used to handle all backend operations, including:

- Receiving input data from the frontend
- Validating and preprocessing the input features
- Loading the appropriate machine learning model
- Performing prediction using the selected model
- Sending the prediction result back to the frontend

This separation of frontend and backend ensures better scalability, modularity, and maintainability of the system.

5.2 Data Flow

The communication between the frontend and backend follows this sequence:

User Input (Streamlit)

- API Request (FastAPI)
- Data Validation & Processing
- Model Prediction
- Response Sent to Frontend

5.3 API Endpoints

The backend exposes REST API endpoints to handle different operations. The key endpoints used in the system are:

Endpoint	Method	Description
/predict	POST	Accepts input features and returns predicted sales value.
/health	GET	Checks the status of the API and confirms if the service is running.

5.4 Integration Outcome

The integration between the frontend and backend allows the system to perform real-time predictions efficiently. The frontend remains lightweight while all computational and model-related tasks are handled by the backend API.

This architecture enables flexibility, as the backend can be scaled or modified independently without affecting the frontend interface.

6. Machine Learning Models

The system utilizes three advanced machine learning models to perform demand prediction. These models were trained during earlier phases and are integrated into the application to provide accurate and reliable predictions.

Each model has different characteristics and strengths, allowing the system to offer flexibility and comparative analysis.

6.1 Random Forest

Random Forest is an ensemble learning algorithm based on the bagging technique. It builds multiple decision trees using random subsets of the data and combines their outputs to improve prediction accuracy.

Key characteristics:

- Handles non-linear relationships effectively
- Reduces overfitting through averaging
- Works well with structured tabular data

In this project, Random Forest performs well in capturing demand patterns and provides stable predictions.

6.2 XGBoost

XGBoost (Extreme Gradient Boosting) is an advanced boosting algorithm that builds trees sequentially, where each new tree corrects the errors of the previous ones.

Key characteristics:

- High performance and efficiency
- Handles complex feature interactions
- Includes regularization to prevent overfitting

XGBoost is particularly effective for structured datasets and improves prediction accuracy by learning residual patterns.

6.3 Neural Network

A Neural Network is a deep learning model that consists of multiple layers of interconnected neurons. It is capable of learning complex and non-linear relationships in the data.

Key characteristics:

- Captures complex patterns in large datasets
- Learns hierarchical feature representations
- Suitable for advanced predictive modeling

In this project, the neural network is trained using multiple layers and optimization techniques to predict continuous sales values.

6.4 Model Usage in the System

The application allows users to select any of the three models for prediction. Based on the selected model:

- The corresponding trained model file (.pkl or .h5) is loaded
- Input data is passed to the model
- Predictions are generated and displayed in real time

Using multiple models enhances the robustness of the system and provides flexibility for users to compare results.

7. Input Features Table

The machine learning models use a set of input features related to supply chain operations to predict sales demand. These features are provided by the user through the frontend interface.

The following table describes the input features used in the system:

Feature Name	Description
Order Item Quantity	Number of items in a particular order.
Shipping Delay (days)	Number of days delayed in shipping.
Profit Margin (%)	Percentage of profit for the order.
Order Month	Month in which the order was placed (captures seasonality)
Order Day Type (Weekend Indicator)	Indicates whether the order was placed on a weekday or weekend.

These features represent important factors that influence demand in a supply chain system. For example:

- Higher order quantities generally indicate increased demand
- Shipping delays can negatively impact sales performance
- Profit margins influence product pricing and demand behavior
- Seasonal patterns (month) affect purchasing trends
- Weekend indicators help capture variation in customer behavior

These inputs are processed and passed to the selected machine learning model to generate the predicted sales output.

8. Deployment

The backend of the Supply Chain Demand Prediction System is designed to be deployment-ready using Docker, enabling consistent and scalable execution across different environments. The application is deployed using Docker Compose, which automates container building and service orchestration.

8.1 Docker-Based Deployment

Docker is used to containerize the backend application, ensuring that all dependencies, libraries, and configurations are packaged together. This allows the application to run reliably regardless of the system on which it is deployed.

Key advantages of using Docker:

- Environment consistency across different systems
- Easy deployment and portability
- Simplified dependency management
- Scalability for production environments

8.2 Application Execution

The system can be executed locally by running both the frontend and backend components:

- Frontend:
  The Streamlit application is executed using: streamlit run app.py
- Backend:
  The FastAPI service can be run either directly or through a Docker container, depending on the setup.

8.3 System Integration

The deployment setup ensures smooth interaction between all components:

- The frontend communicates with the backend API
- The backend processes requests and interacts with machine learning models
- Predictions are generated and stored in the local database
- Results are displayed in real time on the frontend

The use of Docker ensures that the system is portable, scalable, and suitable for deployment in real-world environments.

9. Monitoring

Monitoring is an important aspect of the system that helps track application usage, model performance, and prediction trends over time. It ensures that the system remains reliable and provides insights for future improvements.

9.1 Prediction Monitoring

The system maintains a record of predictions using a local database (predictions.db). This allows the application to track:

- Total number of predictions made
- Average predicted sales
- Peak predicted sales values
- Model usage frequency

These metrics are displayed on the dashboard to provide a clear overview of system usage.

9.2 Dashboard-Based Monitoring

The Streamlit dashboard (dashboard.py) acts as a monitoring interface by visualizing key metrics and trends. It helps users understand how the system is performing and how predictions vary over time.

The dashboard provides:

- Summary statistics of predictions
- Visual representation of model usage
- Insights into prediction patterns

9.3 Importance of Monitoring

Monitoring plays a crucial role in maintaining the effectiveness of the system by:

- Identifying changes in prediction patterns
- Detecting potential model degradation
- Supporting data-driven decision-making
- Enabling future improvements and retraining

9.4 Future Monitoring Scope

In advanced deployments, monitoring can be enhanced using external tools such as Grafana or Power BI to track:

- Data drift
- Model performance degradation
- Real-time system metrics

Overall, monitoring ensures that the system remains reliable, transparent, and adaptable to changing data conditions.

10. Screenshots

The following screenshots demonstrate the working of the Supply Chain Demand Prediction System, including the user interface, prediction functionality, and dashboard visualization.

10.1 Login Page

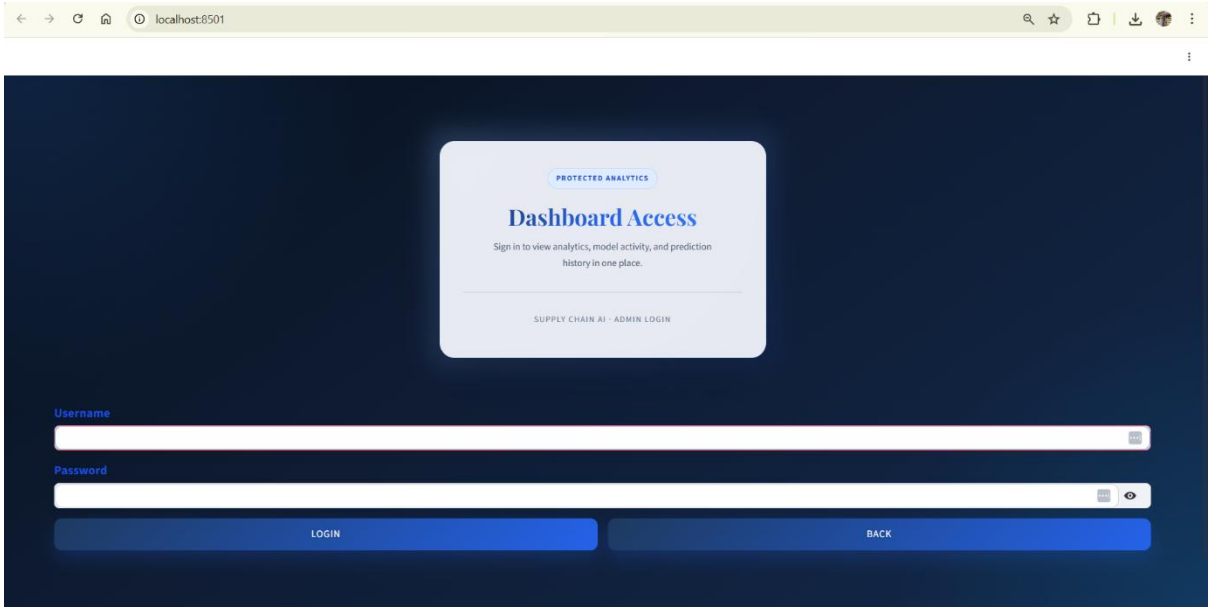


Figure 10.1: Login Page Interface

The login page provides a simple interface for users to access the application. It ensures controlled access and acts as the entry point to the system.

10.2 Prediction Interface

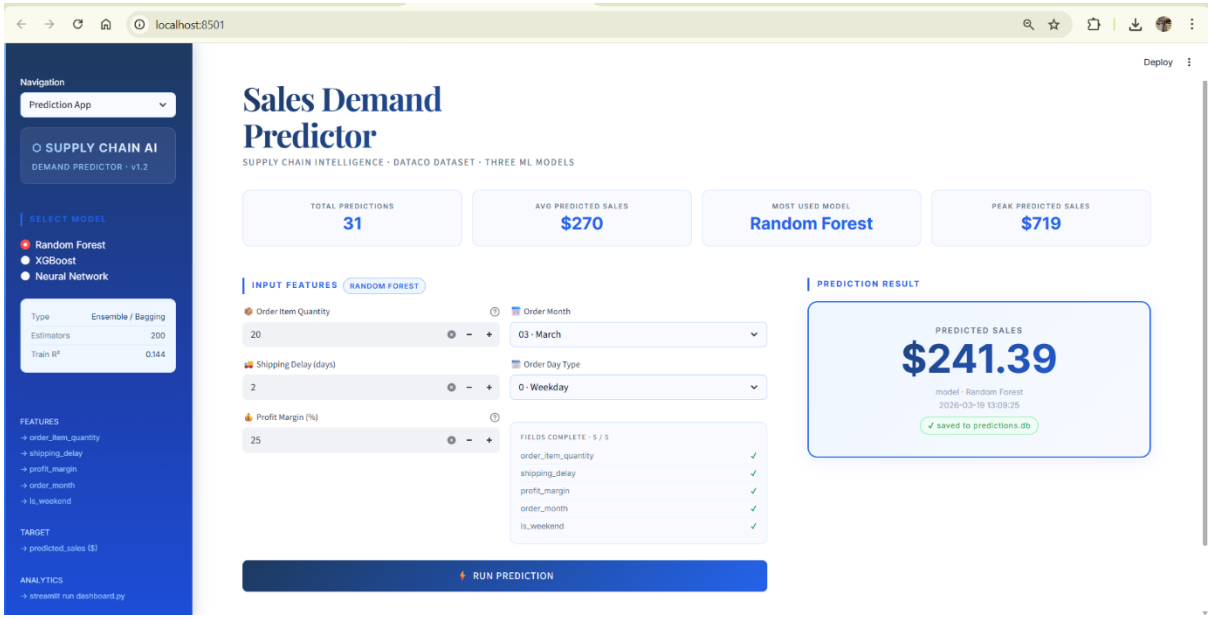


Figure 10.2: Prediction Interface using Random Forest

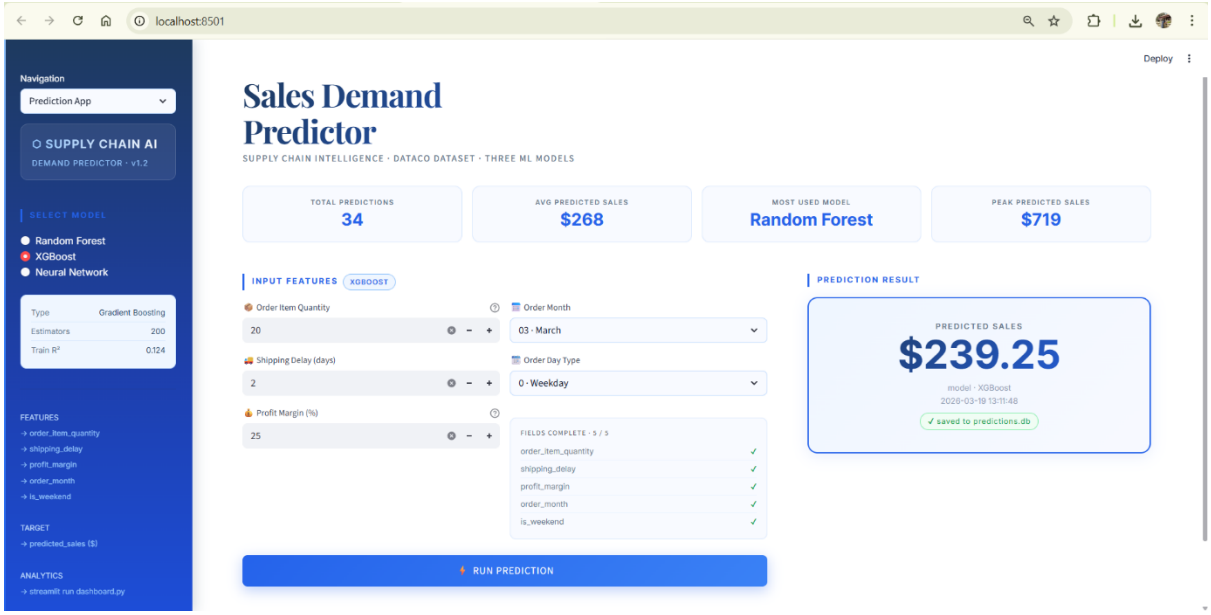


Figure 10.3: Prediction Interface using XGBoost

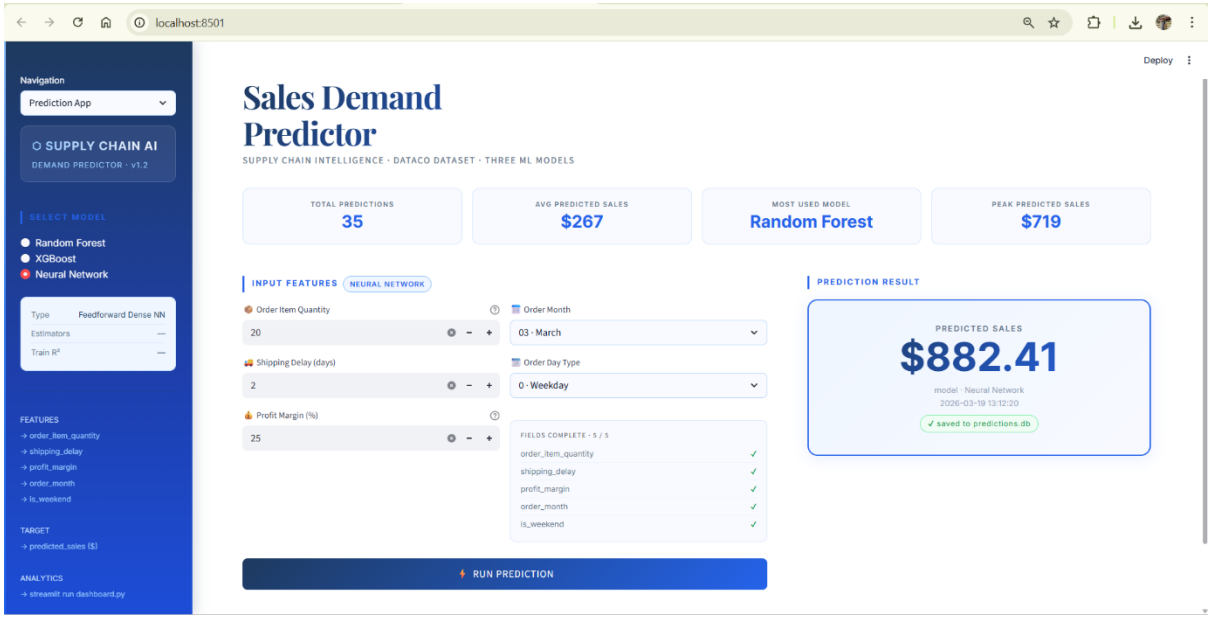


Figure 10.4: Prediction Interface using Neural Network

The prediction interface allows users to input relevant features such as order quantity, shipping delay, profit margin, and seasonal indicators. Users can also select different machine learning models to generate real-time predictions.

10.3 Dashboard View

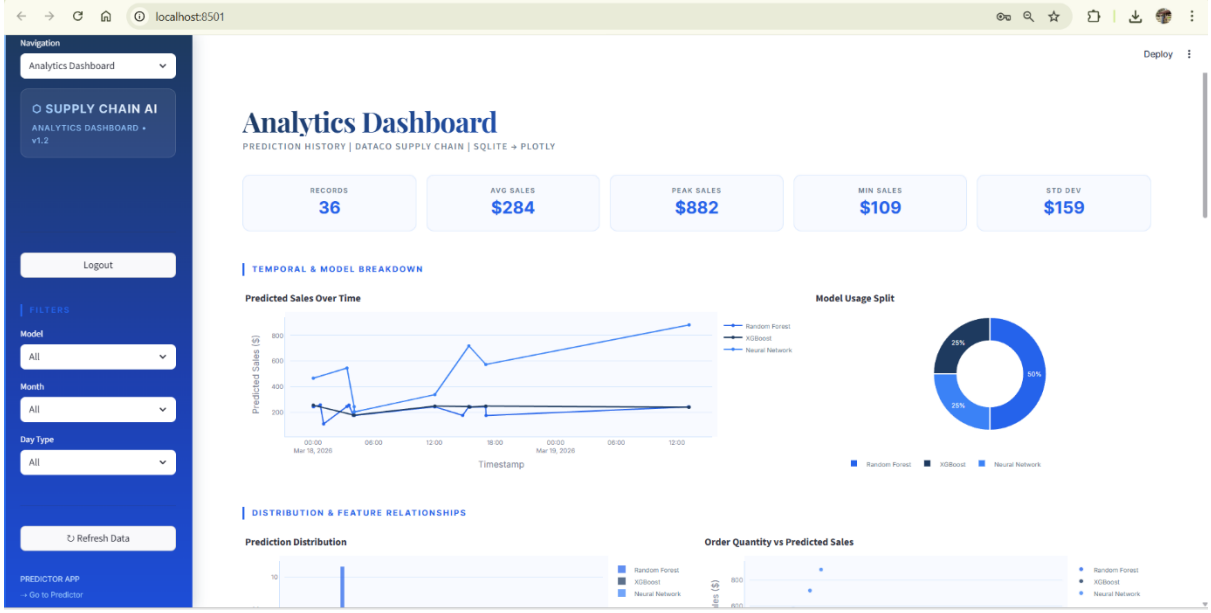


Figure 10.5: Analytics Dashboard Overview with Key Metrics

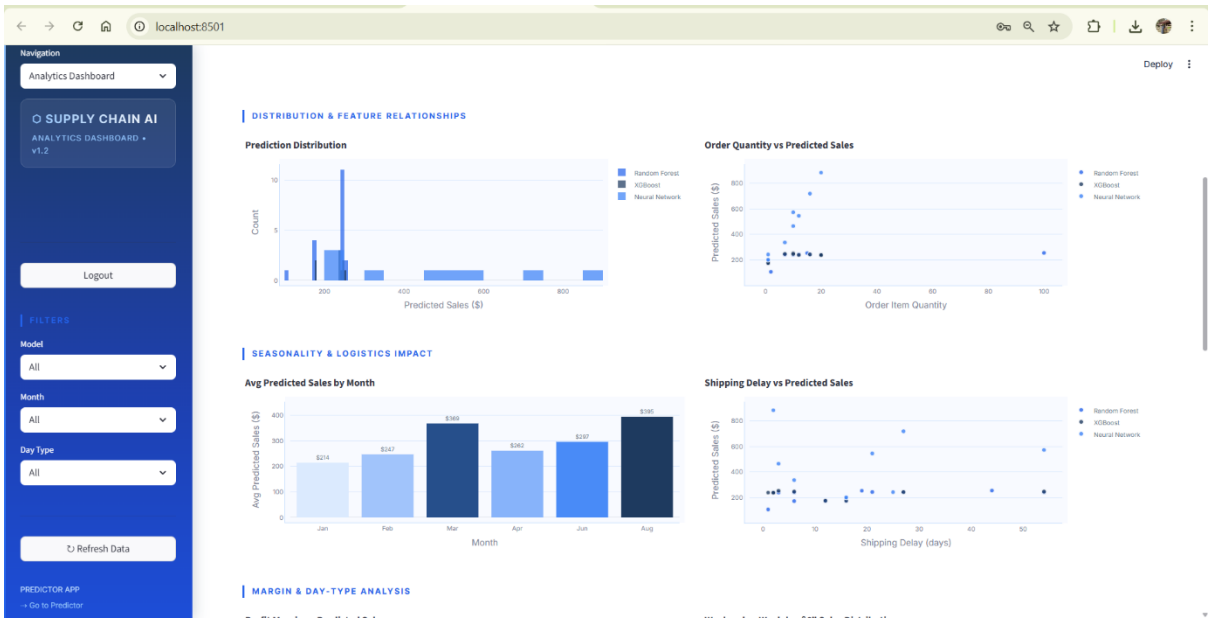


Figure 10.6: Temporal Trends and Model Comparison

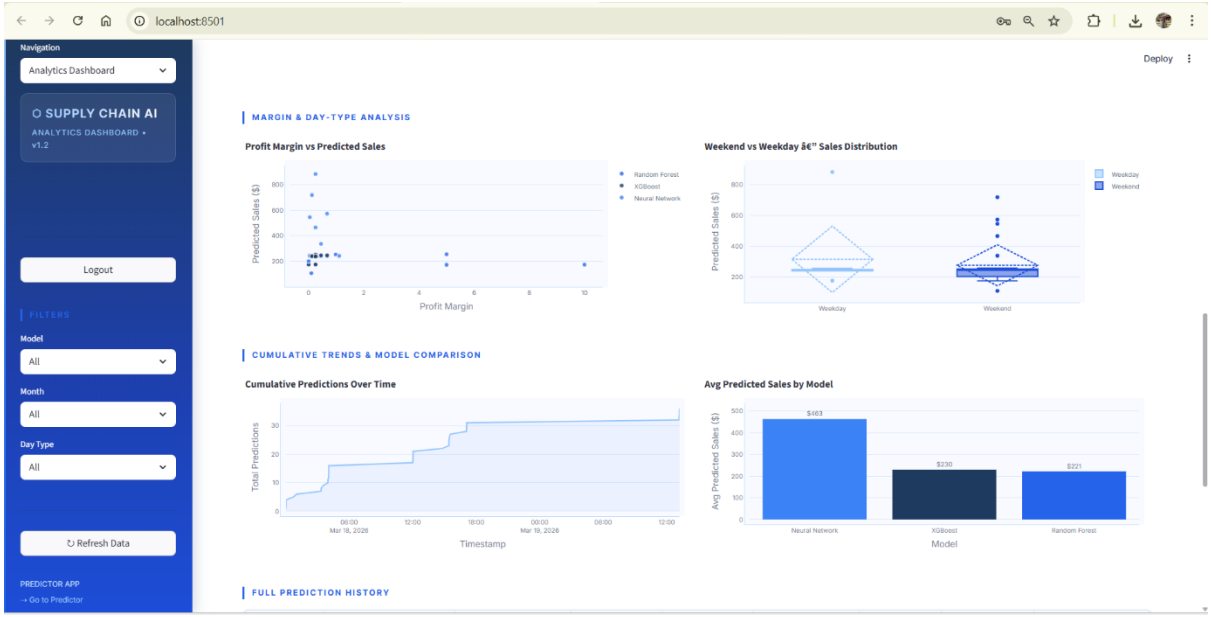
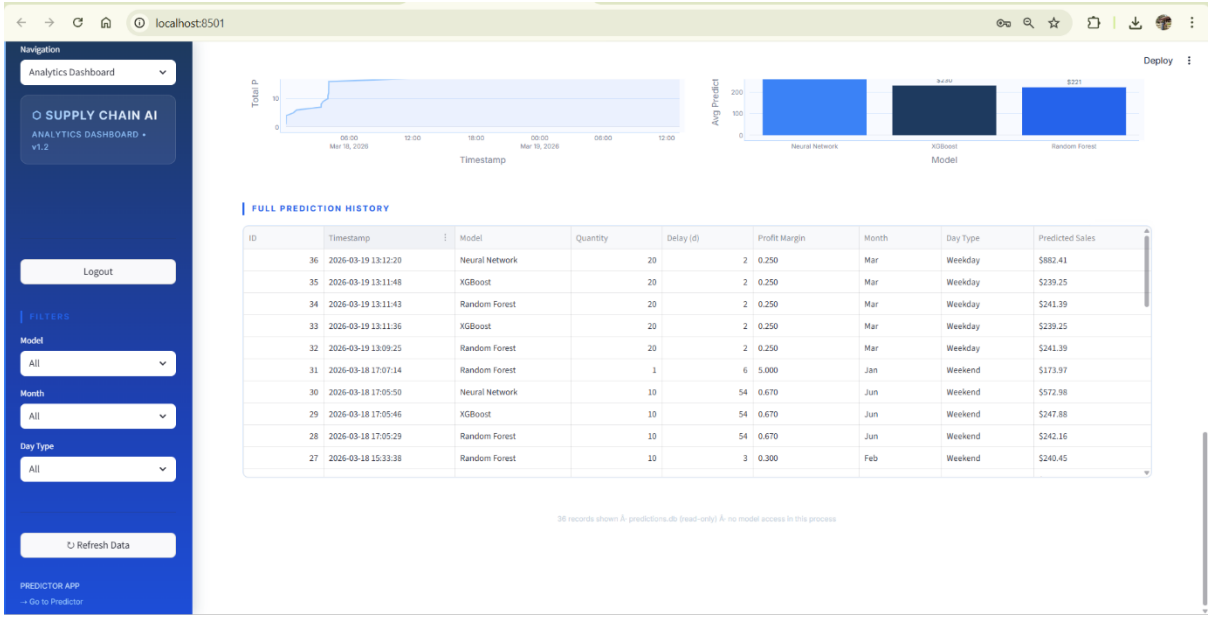


Figure 10.7: Feature Relationships and Prediction Distribution



ID	Timestamp	Model	Quantity	Delay (d)	Profit Margin	Month	Day Type	Predicted Sales
36	2026-03-19 13:12:20	Neural Network	20	2	0.250	Mar	Weekday	$882.41
35	2026-03-19 13:11:48	XGBoost	20	2	0.250	Mar	Weekday	$239.25
34	2026-03-19 13:11:43	Random Forest	20	2	0.250	Mar	Weekday	$241.39
33	2026-03-19 13:11:36	XGBoost	20	2	0.250	Mar	Weekday	$239.25
32	2026-03-19 13:09:25	Random Forest	20	2	0.250	Mar	Weekday	$241.39
31	2026-03-18 17:07:14	Random Forest	1	6	5.000	Jan	Weekend	$173.97
30	2026-03-18 17:05:50	Neural Network	10	54	0.670	Jun	Weekend	$572.98
29	2026-03-18 17:05:46	XGBoost	10	54	0.670	Jun	Weekend	$247.88
28	2026-03-18 17:05:29	Random Forest	10	54	0.670	Jun	Weekend	$242.16
27	2026-03-18 15:33:38	Random Forest	10	3	0.300	Feb	Weekend	$240.45

Figure 10.8: Prediction History and Filtered Data View

The analytics dashboard provides comprehensive insights into prediction results by visualizing trends, model performance, and feature relationships. It includes time-series analysis, model comparison charts, distribution plots, and feature correlation graphs. The dashboard also supports filtering based on model, month, and day type, enabling users to explore prediction patterns effectively. Additionally, the prediction history table allows users to analyze past records stored in the database.

11. Conclusion

The Supply Chain Demand Prediction System successfully demonstrates the integration of machine learning models into a complete, user-friendly application. This phase focuses on transforming the predictive models developed in earlier stages into a functional system that supports real-time decision-making.

The system combines a Streamlit-based frontend, a FastAPI backend, and pre-trained machine learning models to provide accurate demand predictions. Users can interact with the application through an intuitive interface, input relevant features, and obtain predictions instantly.

The inclusion of multiple machine learning models such as Random Forest, XGBoost, and Neural Networks enhances the flexibility and robustness of the system. Additionally, the use of a local database to store prediction history enables the generation of meaningful analytics through the dashboard.

The modular architecture, along with deployment support using Docker, ensures that the system is scalable and can be extended for real-world applications. Monitoring capabilities further strengthen the system by providing insights into usage patterns and performance trends.

Overall, the project demonstrates the practical application of machine learning in supply chain management, highlighting the importance of integrating predictive models with application development and system design.

12. Future Improvements

While the current system provides a functional and integrated solution for demand prediction, there are several areas where further enhancements can be made to improve performance, scalability, and usability.

12.1 Advanced Authentication

Implement a secure authentication system with user roles and access control to ensure data security and personalized user experiences.

12.2 Cloud Deployment

Deploy the application on cloud platforms such as AWS, Azure, or Google Cloud to enable scalability, high availability, and remote accessibility.

12.3 Automated Model Retraining

Introduce automated retraining pipelines to update models periodically based on new data, ensuring improved accuracy and adaptability to changing trends.

12.4 Enhanced Monitoring

Integrate advanced monitoring tools such as Grafana and Prometheus to track real-time system performance, detect anomalies, and monitor data drift and model degradation.

12.5 Improved User Interface

Enhance the frontend design with advanced visualizations, improved layout, and better user experience for more intuitive interaction.

12.6 API Expansion

Extend the API with additional endpoints for analytics, reporting, and model comparison to improve system functionality.

Overall, these improvements can transform the system into a more robust, scalable, and production-ready solution suitable for enterprise-level applications.